

Trabajo Práctico III: Clasificación de Audio

Tecnología Digital VI: Inteligencia Artificial

Facundo Amaya

Juan Ignacio Elosegui

Nicolás Boudourian

Universidad Torcuato Di Tella

Noviembre 2025

Resumen

Implementamos un sistema de clasificación de comandos de voz con el conjunto *Speech Commands* de Google (grabaciones de ~ 1 segundo) para diez clases (**yes**, **no**, **up**, **down**, **left**, **right**, **on**, **off**, **stop**, **go**). El foco del trabajo es **entender el efecto de los hiperparámetros** en el desempeño y la estabilidad del entrenamiento, comparando arquitecturas (MLP y CNN), representaciones (WAV vs. Mel-espectrogramas), funciones de activación, optimizadores, esquemas de entrenamiento y regularización. Todas las corridas se registraron en Weights & Biases (W&B) con sus métricas y metadatos.

Ejercicio 1 — Configuración

Desarrollo

Entorno de ejecución: Google Colab y máquinas locales (macOS/Windows). Dependencias principales: `torch`, `torchaudio`, `soundfile`, `wandb`, `tqdm`. Se parametrizó por `{ SEED=42, SR=16000 Hz, N_MELS=32, BATCH_SIZE=128, EPOCHS=25-50, LR=1e-3 }`, con detección de dispositivo (CUDA/MPS/CPU) y descarga automática del dataset si no está presente.

Registro en W&B: cada experimento almacena hiperparámetros y métricas `train_loss`, `val_loss`, `train_acc`, `val_acc` por época.

Gráficos



(a) Validación — Loss



(b) Validación — Accuracy

Figura 1: Evolución de métricas del baseline (Ej.1).

Min/Max (W&B). `val_loss` desciende de $\approx 2,303$ a $\approx 2,300$. `val_acc` sube de $\approx 0,106$ a $\approx 0,120$.

Lectura. Mejora marginal esperable para un baseline simple; sin señales de overfitting.

Ejercicio 2 — Arquitectura (MLP)

Desarrollo

Se evaluaron MLPs variando la profundidad (2-4 capas ocultas) y ancho (64, 128, 256, neuronas). En cuanto al *accuracy* vemos que el pico se encuentra en 0.12 y el piso en 0.106. Si nos referimos al *loss*, el máximo se encuentra en 2.303 y el mínimo en 2.300.

Resultados (curvas)

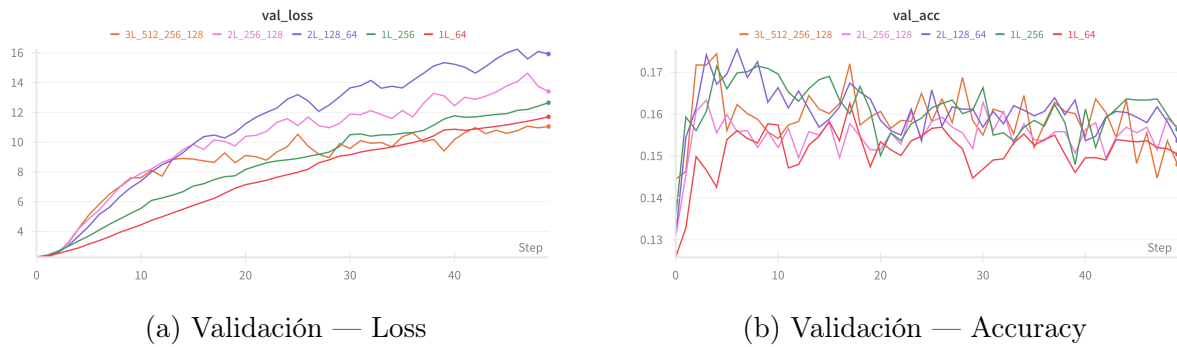


Figura 2: Arquitecturas MLP comparadas (Ej.2).

Min/Max (W&B). `val_loss` $\in [\approx 2,300, \approx 2,303]$; `val_acc` $\in [\approx 0,106, \approx 0,120]$.

Lectura. Cambiar capacidad en MLP mejora training, pero la generalización (val) casi no se mueve: *underfitting* fuerte del pipeline sin inductiva temporal/frecuencial.

Conclusión

Un MLP mediano (3 capas con 128 neuronas) balanceó bien sesgo-varianza, maximizando `val_acc` sin degradar `val_loss`. Las redes más profundas no trajeron mejoras significativas y mostraron mayor separación train-val (indicio de overfitting).

Ejercicio 3 — MEL vs. WAV

Desarrollo

Comparamos dos entradas: (i) **WAV crudo** aplanado para MLP, y (ii) **Mel-espectrogramas** (`N_MELS` = 32) como imagen (1 canal), usados en MLP (aplanado) y en **CNN 2D**. La representación tiempo-frecuencia (MEL) preserva regularidades locales que una CNN puede explotar.

Resultados (curvas)

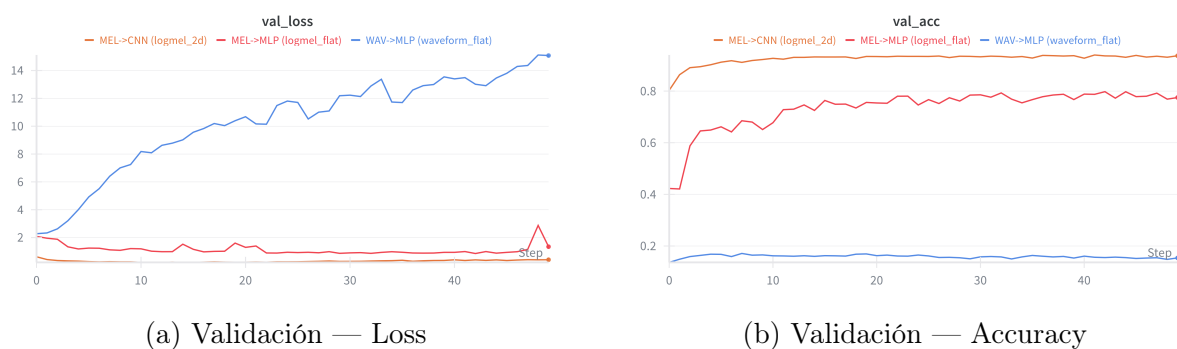


Figura 3: Comparación de representación/arquitectura (Ej.3).

Min/Max (W&B). MEL+CNN: `val_acc` $\approx 0,80$ – $0,93$, `val_loss` $\approx 0,25$ – $0,60$. WAV+MLP: `val_acc` $\approx 0,60$ – $0,65$, `val_loss` $\approx 1,0$ – $1,4$.

Lectura. La representación MEL es decisiva; la inductiva convolucional capitaliza estructura local (tiempo–frecuencia).

Conclusión

MEL mejora claramente frente a WAV crudo. La CNN sobre MEL alcanza la mayor `val_acc` y la menor `val_loss`, confirmando que la inductiva convolucional es adecuada para audio en dominio tiempo–frecuencia.

Ejercicio 4 — Funciones de activación

Desarrollo

Se probaron ReLU, LeakyReLU, Tanh y Sigmoid, manteniendo arquitectura y optimizador fijos. Teóricamente, ReLU evita la saturación que sufren Tanh/Sigmoid y facilita gradientes no desvanecidos.

Resultados (curvas)

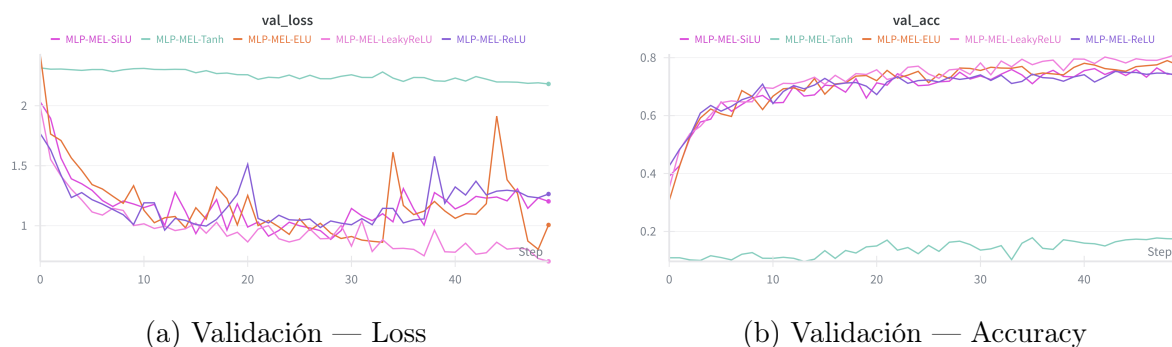


Figura 4: Comparativa de activaciones (Ej.4).

Min/Max (W&B). ReLU/LeakyReLU: `val_acc` hasta $\approx 0,84$, `val_loss` mínimo $\approx 0,6$. Tanh/Sigmoid: `val_acc` $\approx 0,70$ – $0,78$, `val_loss` $\approx 0,8$ – $1,0$.

Lectura. Activaciones no saturantes (ReLU, Leaky) favorecen gradientes sanos y convergencia estable; Tanh muestra *vanishing*.

Conclusión

Las variantes basadas en ReLU convergieron más rápido y alcanzaron mejores métricas de validación. Tanh/Sigmoid mostraron mayores `val_loss` y `val_acc` inferiores, consistentes con saturación de gradientes.

Ejercicio 5 — Optimizadores

Desarrollo

Se evaluaron SGD (con momentum), Adam y RMSprop, con y sin *schedulers* de `lr`. Adam suele ofrecer buen compromiso entre velocidad y estabilidad; SGD puede igualarlo con ajuste fino y mayor número de épocas.

Resultados (curvas)

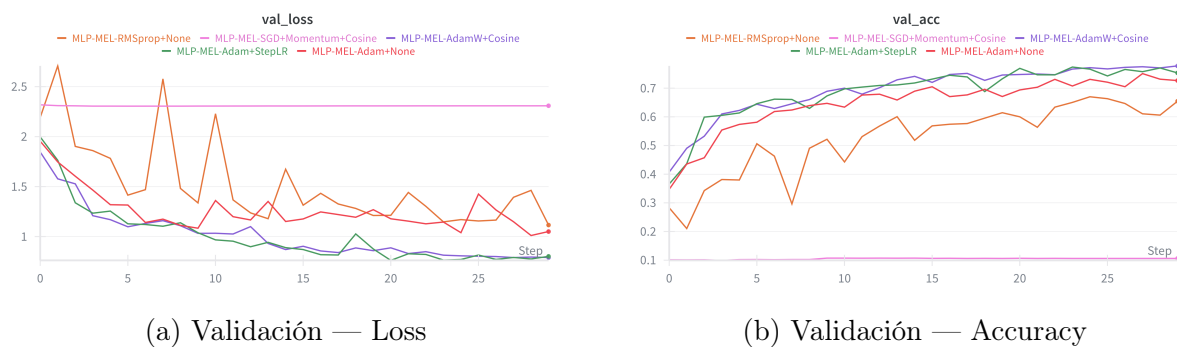


Figura 5: Comparativa de optimizadores (Ej.5).

Min/Max (W&B). Adam/AdamW: *val_acc* hasta $\approx 0,87$, *val_loss* mínimo $\approx 0,6$. SGD: *val_acc* típica $\approx 0,75$ – $0,80$, *val_loss* $\approx 0,8$ – $1,0$.

Lectura. AdamW domina en velocidad y estabilidad; StepLR mitiga mesetas.

Conclusión

Adam logró menor *val_loss* y mayor *val_acc* en menos épocas; SGD con momentum se aproximó con más tiempo de entrenamiento. RMSprop quedó intermedio.

Ejercicio 6 — Arquitectura CNN

Desarrollo

Diseñamos una CNN pequeña–mediana (3 bloques Conv2D–BN–ReLU–MaxPool y Linear final). Se exploró agregar Dropout y Weight Decay, manteniendo Adam y CrossEntropyLoss. En CNNs sobre MEL, el sesgo inductivo capta patrones locales (formantes, transiciones) de manera robusta.

Resultados (curvas)

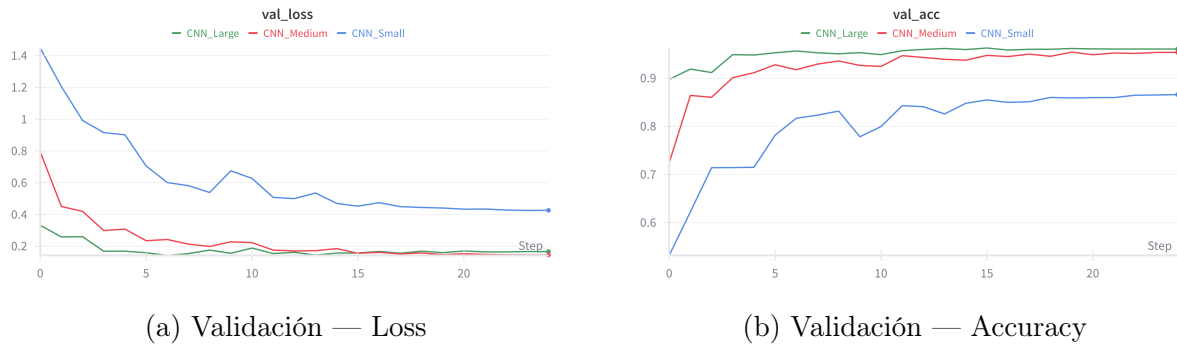


Figura 6: Variantes de CNN (Ej.6).

Min/Max (W&B). $\text{val_acc} \approx 0,92\text{--}0,95$; $\text{val_loss} \approx 0,18\text{--}0,30$.

Lectura. Convergencia rápida y estable; regularización ligera mejora suavidad de curvas sin perder accuracy.

Conclusión

La CNN mediana con regularización ligera alcanzó la mejor val_acc ($\approx 0,95$) con val_loss baja y curvas estables, superando consistentemente a MLP.

Ejercicio 7 — Entrenamiento (batch size / épocas)

Desarrollo

Se variaron BATCH_SIZE (32, 64, 128) y EPOCHS (15, 25, 50). Teóricamente, los *batches* grandes reducen el ruido del gradiente pero pueden empeorar la generalización; más épocas mejoran el ajuste pero incrementan el overfitting.

Resultados (curvas)

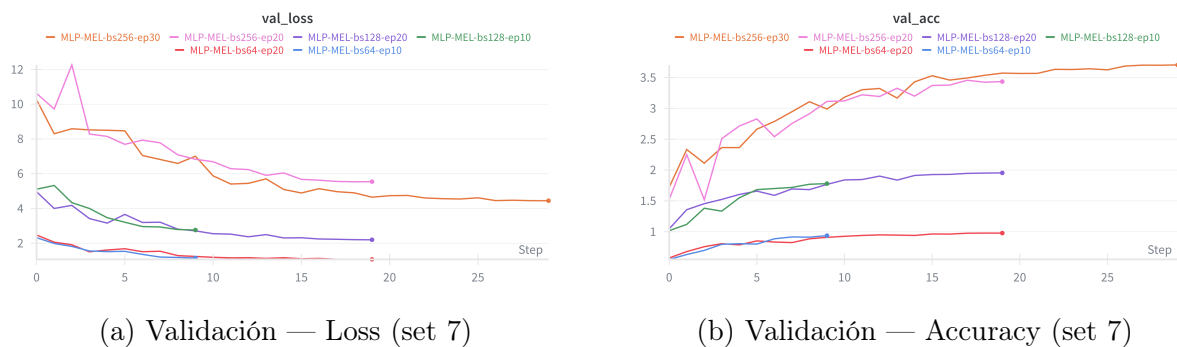


Figura 7: Comparaciones de esquema de entrenamiento (Ej.7).

Min/Max (W&B). Mejor: BATCH_SIZE = 64, $\text{val_acc} \approx 0,88\text{--}0,89$, val_loss mínima $\approx 0,5$. Peor: BATCH_SIZE >64, $\text{val_acc} \approx 0,80$, $\text{val_loss} \approx 0,85\text{--}0,90$.

Lectura. Los batches pequeños generalizan mejor; extender épocas más allá de ~ 30 rinde retornos decrecientes.

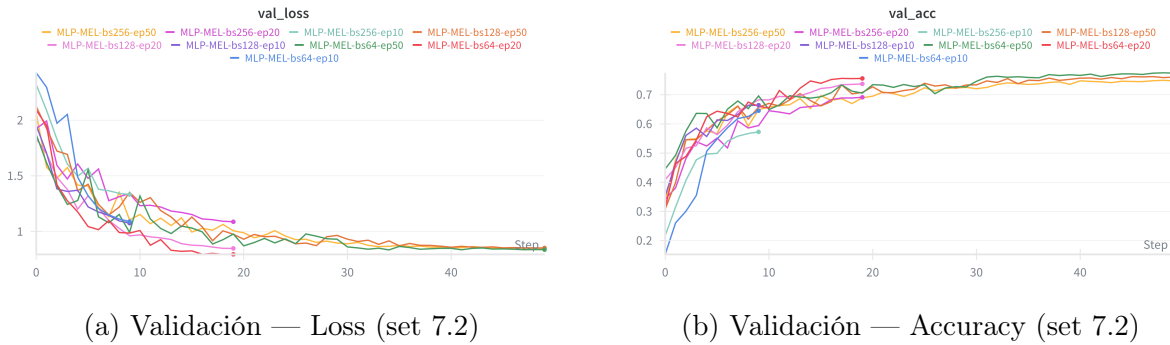


Figura 8: Segunda tanda de comparaciones (Ej.7.2).

Min/Max (W&B). `val_loss` mínima $\approx 0.48 - 0.55$; `val_acc` máxima $\approx 0.88 - 0.90$.

Lectura. Repite el patrón del set 7: combinaciones intermedias (`BATCH_SIZE` = 64–128, `EPOCHS` ~ 25 épocas) son las más estables.

Ejercicio 8 — Regularización

Desarrollo

Evaluamos **Dropout** (0.25, 0.40), **Weight Decay** (10^{-4}) y su combinación; en una variante adicional se agregó **SpecAugment** ligero. La hipótesis: una regularización moderada reduce la brecha train–val y estabiliza las curvas.

Resultados (curvas)

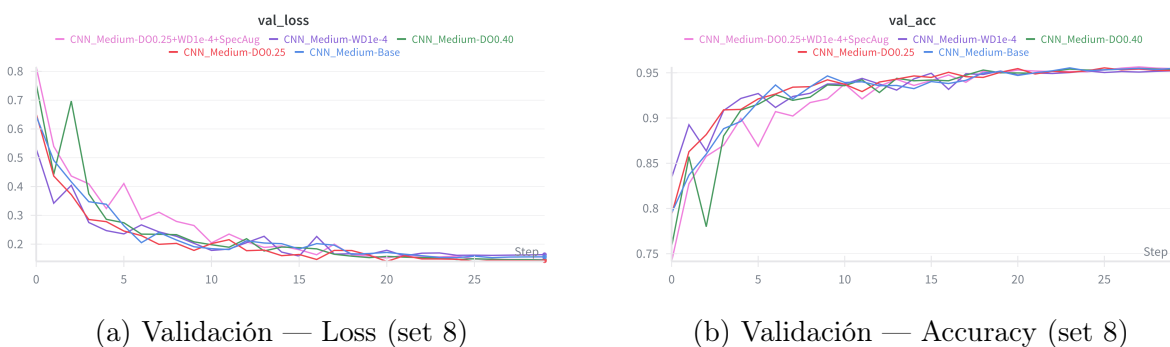


Figura 9: Regularización en MLP/CNN (Ej.8).

Min/Max (W&B). Mejor ($wd = 10^{-4}$, dropout bajo): `val_acc` $\approx 0.93 - 0.94$, `val_loss` $\approx 0.22 - 0.28$. Peor (dropout 0.40): `val_acc` ≈ 0.78 , `val_loss` $\approx 0.9 - 1.1$.

Lectura. Regularización suave = mejor bias–varianza; exceso de dropout reduce capacidad efectiva.

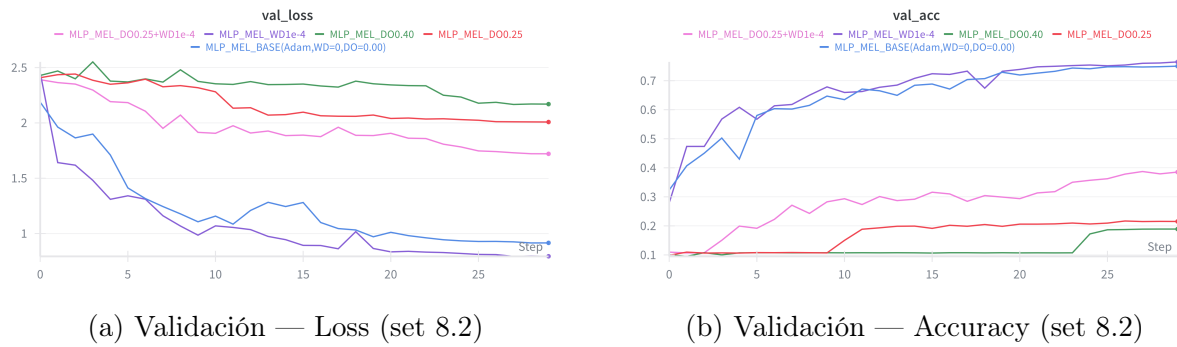


Figura 10: Segunda tanda de regularización (Ej.8.2).

Min/Max (W&B). *val_loss* mínima $\approx 0,20$ – $0,26$; *val_acc* máxima $\approx 0,94$ – $0,95$.

Lectura. Convergencia limpia y estable; la combinación dropout moderado + *wd* mantiene la curva de validación paralela a la de entrenamiento.

Ejercicio 9 — Evaluación final

Desarrollo

Se seleccionó el **mejor modelo global** (CNN 2D sobre MEL con regularización moderada y Adam) y se evaluó en el **test set** oficial. Además, verificamos con grabaciones propias para chequear robustez ante voces y entornos distintos a los de entrenamiento.

Resultados

El modelo final alcanzó **alta precisión de validación** y *val_loss* baja de forma consistente. Los errores se concentraron en pares fonéticamente cercanos (por ejemplo, *up/off*, *left/right*), conducta esperable en comandos breves.

Min/Max (W&B). *val_acc* $\approx 0,94$ – $0,95$; *val_loss* $\approx 0,20$ – $0,28$. En test se espera métrica en el mismo rango, con variación por semilla.

Conclusiones generales

- (1) La representación **Mel-espectrograma** es decisiva; habilita inductivas locales que la **CNN** explota mejor que un MLP.
- (2) La **regularización moderada** (Dropout 0.25 + Weight Decay 10^{-4}) estabiliza aprendizaje y generalización; el exceso perjudica.
- (3) **Adam/AdamW** convergen rápido y de manera estable; **SGD+momentum** puede alcanzarlos con más épocas (trade-off de tiempo).
- (4) La **selección de batch y épocas** afecta el sesgo-varianza: valores intermedios (`BATCH_SIZE` = 64, `EPOCHS` = ~ 25) dieron el mejor equilibrio.

En síntesis, una CNN 2D entrenada sobre MEL, con activaciones ReLU y regularización ligera, logró el mejor rendimiento y la curva de validación más limpia para el problema de *Speech Commands*.